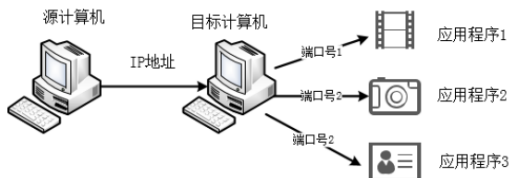


Lecture 8, 9

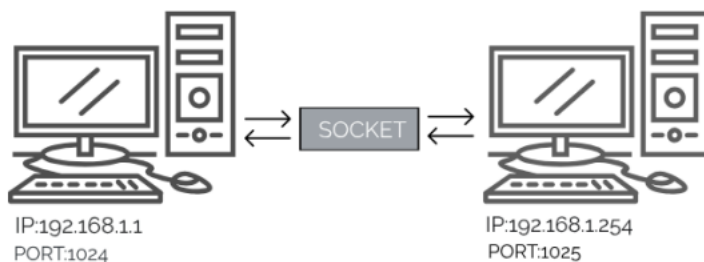
- Socket

- socket programming

- 网络（Networking）：网络是将两个或多个计算设备连接在一起以便共享资源的概念。
 - 网络术语



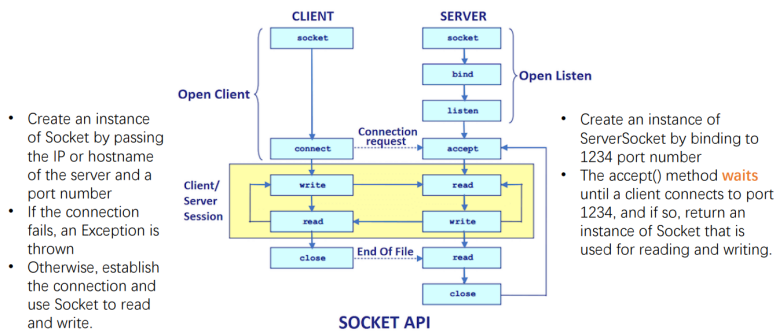
- IP 地址：用于区分互联网上或局域网中设备的唯一地址。
 - 域名：IP 地址的对人友好的表示形式，通常在浏览器中输入，由 DNS 翻译为 IP 地址。
 - 端口号：用于唯一标识不同应用/进程的数字。
 - 套接字（Socket）



- 要通信，客户端程序和服务器程序需要建立相互之间的连接。
 - 每个程序在连接的一端绑定一个套接字（socket）。
 - 套接字是两个在网络上运行的程序之间双向通信链路的一个端点。
 - 端点由：IP 地址 + 端口号 组成。
 - 通信时，客户端和服务器各自对绑定到该连接的套接字进行读写。
 - java.net 包提供了强大且灵活的网络基础设施，包含执行底层通信功能的各种类和接口。
 - 常用构造器示例：
 - Socket(String host, int port)：创建一个流套接字并将其连接到指定主机的指定端口。
 - ServerSocket(int port)：创建绑定到指定端口的服务器套接字。

```
Socket s = new Socket("www.serverip.com", 1234);
```

```
ServerSocket ss = new ServerSocket(1234);
Socket s = ss.accept();
```



- Create an instance of Socket by passing the IP or hostname of the server and a port number
- If the connection fails, an Exception is thrown
- Otherwise, establish the connection and use Socket to read and write.

- Create an instance of ServerSocket by binding to 1234 port number
- The accept() method **waits** until a client connects to port 1234, and if so, return an instance of Socket that is used for reading and writing.

A Toy Example: Client

```
public class SimpleTcpClient {
    public static void main(String[] args) throws IOException {
        // connect to localhost's 1234 port
        // return a socket if the connection succeeds
        Socket socket = new Socket("localhost", 1234);

        // Get OutputStream
        // and write messages as bytes
        OutputStream os = socket.getOutputStream();
        byte[] msg = "Hello server!".getBytes();
        os.write(msg);
        System.out.println("Client's message sent");

        // closing the OutputStream will close the associated socket.
        os.close();
    }
}
```

A Toy Example: Server

```
public class SimpleTcpServer {
    public static void main(String[] args) throws IOException {
        // Listen to port 1234
        ServerSocket serverSocket = new ServerSocket(1234);

        // accept() blocks until a client connects
        // if a client connects successfully, return a Socket instance
        System.out.println("Waiting for client.....");
        Socket socket = serverSocket.accept();
        System.out.println("Client connected.");

        // use the socket's inputStream to read message from the client
        InputStream inputStream = socket.getInputStream();

        // get client msg as bytes and print it
        byte[] buf = new byte[1024];
        int readLen = 0;
        while((readLen = inputStream.read(buf)) != -1){
            System.out.println(new String(buf, 0, readLen));
        }

        // closing the InputStream will close the associated socket
        inputStream.close();
        serverSocket.close();
    }
}
```

Case Study: Banking Service

BankService
BankService(Socket, Bank)
run(): void
socket: Socket
bank: Bank

A bank service executes the banking service protocol

```
public class BankService implements Runnable {
    private Socket socket;
    private Bank bank;

    public BankService (Socket socket, Bank bank) {
        this.socket = socket;
        this.bank = bank;
    }
}
```

Case Study: Banking Service

BankService
BankService(Socket, Bank)
run(): void
socket: Socket
bank: Bank

```
public void run() {
    Scanner in = new Scanner( socket.getInputStream());
    PrintWriter out = new PrintWriter( socket.getOutputStream());
```

```
while (true) {
    if (!in.hasNext()) return;
    String command = in.next();
    if ("QUIT".equals(command)) return;
```

```
int account = in.nextInt();
double amount;
switch (command) {
    case "DEPOSIT":
        amount = in.nextDouble();
        bank.deposit( account, amount);
        break;
    case "WITHDRAW":
        amount = in.nextDouble();
        bank.withdraw( account, amount);
        break;
    case "BALANCE":
        break;
    default:
        out.println( "Invalid command" );
        out.flush();
        return;
}
```

```
out.println( account + " " + bank.getBalance( account) );
out.flush();
}
```

Table 2 A Simple Bank Access Protocol

Client Request	Server Response	Description
BALANCE <i>n</i>	<i>n</i> and the balance	Get the balance of account <i>n</i>
DEPOSIT <i>n a</i>	<i>n</i> and the new balance	Deposit amount <i>a</i> into account <i>n</i>
WITHDRAW <i>n a</i>	<i>n</i> and the new balance	Withdraw amount <i>a</i> from account <i>n</i>
QUIT	None	Quit the connection

22

Bank Server

```
public class BankServer {
    public static void main (String[] args) throws IOException {
        // initialized bank with 10 accounts
        Bank bank = new Bank(10);

        ServerSocket server = new ServerSocket(8888);

        System.out.println( "Waiting for clients to connect..." );
        // server keeps running
        while (true) {
            Socket socket = server.accept();
            System.out.println( "Client connected." );

            // start a thread for providing service to the client
            BankService service = new BankService(socket, bank);
            Thread t = new Thread(service);
            t.start();
        }
    }
}
```

Bank Client

- To communicate with the server by sending and receiving text, you could turn the streams into **Scanners** and **PrintWriters**
- Use **println** instead of **print** to mark the end of input
- The **flush()** method empties the buffer and forwards all waiting characters to the destination.

```
public class BankClient {
    public static void main (String[] args) throws IOException {

        Socket s = new Socket("localhost", 8888);

        Scanner in = new Scanner(s.getInputStream());
        PrintWriter out = new PrintWriter(s.getOutputStream());

        String command = "DEPOSIT 3 1000";
        System.out.println("Sending: " + command);

        out.println(command);
        out.flush();

        String response = in.nextLine();
        System.out.println("Receiving: " + response);

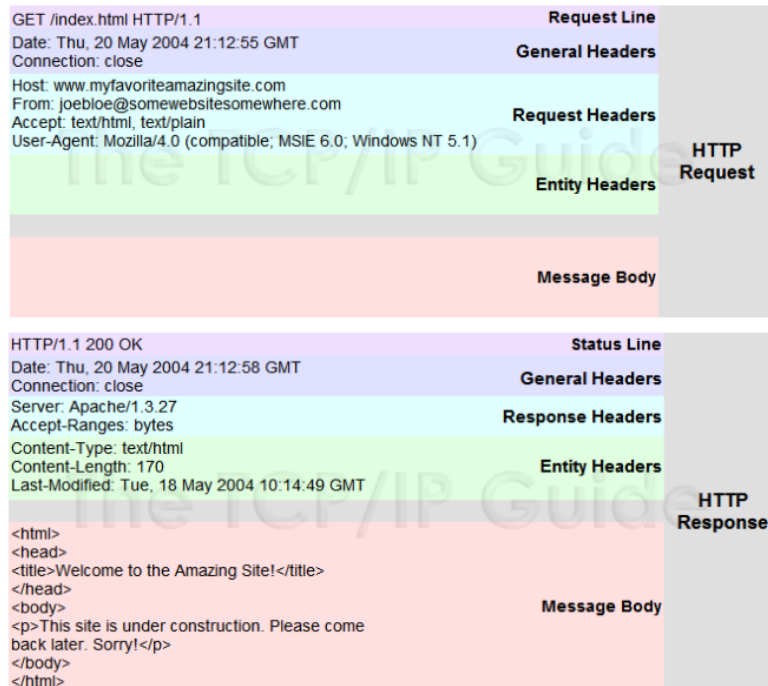
        command = "QUIT";
        out.println(command);
        out.flush();

        s.close();
    }
}
```

• getting web data

- 应用层协议（Application Layer Protocols）：
 - 每个互联网应用有不同的应用协议，该协议定义了该具体应用的数据如何被传输。
 - 端口号帮助计算机决定哪个应用程序应接收传入的数据。
 - 常见端口号示例（端口号：使用该端口的协议）：
 - 21：文件传输协议（FTP）
 - 25：简单邮件传输协议（SMTP）
 - 80 & 8080：超文本传输协议（HTTP）
 - 110：邮局协议 v3（POP3）
 - 143：互联网邮件访问协议（IMAP）
 - 443：基于 SSL/TLS 的超文本传输协议（HTTPS）
 - 989：安全 FTP（SFTP）
 - 23：Telnet
 - 注意：知名端口号（well-known ports）通常被保留，不能随意用于其它用途。
- HTTP（超文本传输协议）
 - HTTP 是一种用于获取资源（例如 HTML 文档）的协议，是 Web 上任意数据交换的基础。
 - 它是一个客户端-服务器协议，意味着请求由客户端发起，通常是网页浏览器。
 - Web 服务器以 HTTP 响应进行回复。
 - 常用命令及含义：
 - GET：返回请求的资源
 - HEAD：只请求资源的头部信息
 - OPTIONS：请求资源的通信选项
 - POST：向服务器端命令提交输入并返回结果

- PUT：在服务器上存储资源
- DELETE：删除服务器上的资源
- TRACE：跟踪服务器通信
- HTTP Request/Response Message Format



- 使用套接字获取网页

```
public static void readBySocket(String host) throws IOException {
    try(Socket s = new Socket(host, 80)){
        InputStream instream = s.getInputStream();
        OutputStream outstream = s.getOutputStream();

        // working with text
        Scanner in = new Scanner(instream);
        PrintWriter out = new PrintWriter(outstream);

        // Send command
        String resource = "/";
        String command = "GET " + resource + " HTTP/1.1\n" +
            "Host: " + host + "\n";
        System.out.println(command);
        out.println(command);
        out.flush();

        // Read server response
        while(in.hasNextLine()){
            String input = in.nextLine();
            System.out.println(input);
        }
    }
}
```

- 问题：
 - 我们必须自己处理套接字连接和套接字错误
 - 我们必须手动按照正确格式创建 HTTP 请求

- 我们必须手动解析 HTTP 响应
- 为了在 Java 中访问 Web 服务器，我们希望在比套接字连接和 HTTP 请求更高的抽象层次上工作
- java.net 包——提供用于实现网络应用程序的类

- java.net 包大致可以分为两个部分：

- 低级 API，处理以下抽象：
 - 地址：网络标识符，例如 IP 地址
 - 套接字：基本的双向数据通信机制
 - 接口：描述网络接口
- 高级 API，处理以下抽象：
 - URI：通用资源标识符
 - URL：通用资源定位符

```
public static void readByURLConnection(String url) throws IOException {
    URL u = new URL(url);
    // Open connection
    URLConnection conn = u.openConnection();
    // For HTTP an HttpURLConnection will be returned
    HttpURLConnection httpConn = (HttpURLConnection) conn;

    // Check response code and status
    int code = httpConn.getResponseCode();
    String msg = httpConn.getResponseMessage();
    System.out.println(code + " " + msg);
    if (code != HttpURLConnection.HTTP_OK) {
        return;
    }

    // Read server response
    InputStream istream = httpConn.getInputStream();
    Scanner in = new Scanner(istream);
    while (in.hasNextLine()) {
        System.out.println(in.nextLine());
    }
}
```

- 连接：表示与 URL 指向的资源的连接
- REST API
 - REST API 的运行
 - REST 客户端（例如浏览器或移动应用）
 - REST 请求：HTTP 方法 + 资源标识符（URI）
 - 常用方法：GET、POST、PUT、DELETE
 - REST 响应：以 XML 或 JSON 格式表示的资源
 - REST 服务器及其上的各类资源
 - RESTful API 请求设计
 - 请求 = 动词 + 资源
 - 常用 HTTP 动词：GET、PUT、PATCH、POST、DELETE
 - 设计原则：

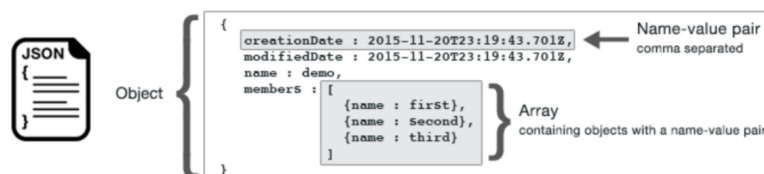
- 通常使用复数名词表示资源，例如：questions
- 允许使用参数对结果进行筛选，例如：?limit=10
- 常见端点示例及说明：

- `GET /repos/{owner}/{repo}`
 - 根据仓库所有者和仓库名称获取仓库信息
- `POST /repos/{owner}/{repo}/issues`
 - 创建一个 issue（需要对该仓库具有相应权限）
- `PATCH /repos/{owner}/{repo}/releases/{release_id}`
 - 更新发布（需要对该仓库具有推送权限）

- Stack Overflow REST API

REST Service URL	Requested resource	Parameter
<pre>String s= "https://api.stackexchange.com/questions/27872387?site=stackoverflow"; URL url = new URL(s); URLConnection conn = (URLConnection)url.openConnection(); conn.setRequestMethod("GET"); conn.connect(); int responseCode = conn.getResponseCode(); String responseMessage = conn.getResponseMessage(); String contentEncoding = conn.getContentEncoding();</pre>		
	java.net package	
	Request verb	
	200	
	OK	
	gzip	

- REST 服务 URL、请求的资源、以及参数三部分组成请求
- 使用 java.net 包创建 URL：new URL(s)
- 打开连接并转换为 HttpURLConnection：(HttpURLConnection) url.openConnection()
- 设置请求方法（动词），例如 conn.setRequestMethod("GET")
- 发起连接：conn.connect()
- 获取响应信息：
 - conn.getResponseCode()（例如 200）
 - conn.getResponseMessage()（例如 OK）
 - conn.getContentEncoding()（例如 gzip）
- 响应通常以 JSON 格式返回



- HTTP 状态码类别：
 - 1xx：信息性响应

- 2xx: 成功
- 3xx: 重定向
- 4xx: 客户端错误
- 5xx: 服务器错误

• JavaFX

• 概述

- JavaFX 是一个开源的、面向下一代的客户端应用平台，适用于桌面、移动和嵌入式系统，基于 Java（即 Java 的 GUI 工具包）。
- JavaFX 可运行于多种操作系统和设备：Windows, Linux, macOS, iOS, Android / Chromebook, 树莓派（Raspberry Pi）

• Hello World

•

```
import javafx.application.Application;
import javafx.stage.Stage;

public class MyFxApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        primaryStage.setTitle("My First JavaFX App");
        primaryStage.show();
    }

    public static void main(String[] args) {
        Application.launch(args);
    }
}
```

Import necessary classes from **javafx**

Extend the abstract **Application** class

Implement the abstract **start()** method of the **Application** class (called when a JavaFX application starts)

launch() launches the JavaFX application.

Optional

- 从 javafx 导入所需的类
 - 继承抽象类 Application
 - 实现 Application 的抽象方法 start(Stage primaryStage)，该方法在 JavaFX 应用启动时被调用
 - 在 start 方法中设置窗口标题（例如 primaryStage.setTitle(...)）
 - 调用 primaryStage.show() 使应用窗口可见（否则不会显示任何内容）
 - 可选：在主方法 public static void main(String[] args) 中调用 Application.launch(args) 来启动 JavaFX 应用（launch() 用于启动应用）
- JavaFX Hello World


```

@Override
public void start(Stage primaryStage) throws Exception {
    primaryStage.setTitle("My First JavaFX App");

    StackPane root = new StackPane();

    Button btn = new Button();
    btn.setText("Hello World");
    btn.setOnAction(new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent event) {
            System.out.println("Hello World!");
        }
    });

    root.getChildren().add(btn);

    Scene scene = new Scene(root, width: 400, height: 200);
    primaryStage.setScene(scene);

    primaryStage.show();
}

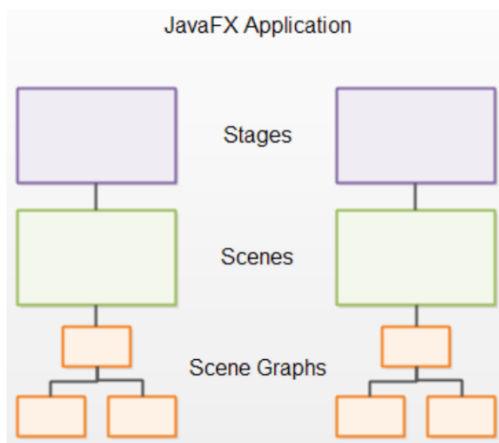
```

- 步骤说明：

- 根节点使用可调整大小的布局节点（例如 StackPane）。
- 子节点为一个按钮，并为按钮设置事件处理器，按下时执行相应操作（例如打印消息）。
- 将按钮添加到根节点的子节点集合中。
- 使用根节点创建一个场景（可指定宽高）。
- 将场景设置到舞台并调用显示方法使窗口可见。

- 设计与概念

- JavaFX 设计



- Stage（窗体）

- JavaFX 应用的外部框架，通常对应一个窗口。
- 一个 JavaFX 应用可以有一个或多个 Stage（可同时打开多个窗口）。

- 当 JavaFX 应用启动时，会创建一个 Stage 对象并传入 start(Stage primaryStage) 方法。

```
import javafx.application.Application;
import javafx.stage.Stage;

public class MyFxApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        primaryStage.setTitle("My First JavaFX App");

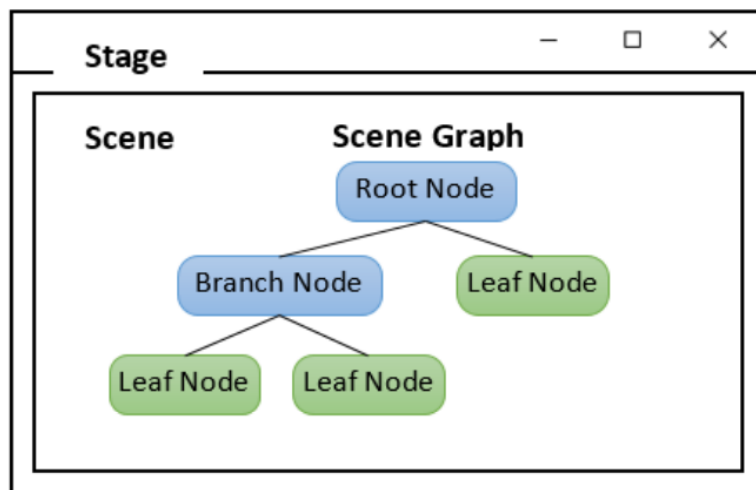
        primaryStage.show();
    }

    public static void main(String[] args) {
        Application.launch(args);
    }
}
```

New stage could be created by:

```
Stage stage = new Stage();
.....
stage.show()
```

- JavaFX 窗体模态 (Stage Modality)
 - 窗体的模态性决定该窗体是否会阻塞同一 JavaFX 应用中打开的其他窗口。
 - 模态窗体通常用于阻止用户在完成当前交互（例如对话框输入）前访问其他窗口。
 - APPLICATION_MODAL：定义一个模态窗口，会阻止事件传递到任何其他应用程序窗口。
 - NONE：定义一个顶层非模态窗口，不阻塞任何其他窗口。
 - WINDOW_MODAL：定义一个模态窗口，会阻止事件传递到其所属窗口层次（所属窗口及其子窗口）。
- Scene (场景)
 - 包含窗口中所有可见的 GUI 组件（即显示在 Stage 上的内容）。
 - 一个 Scene 同一时间只能附加到一个 Stage，Stage 也只能同时显示一个 Scene。
 - Scene 对象通过指定一个根 GUI 组件（场景图中的根节点）来创建。
 - 一个 JavaFX 场景包含其中的所有可视 JavaFX GUI 组件。

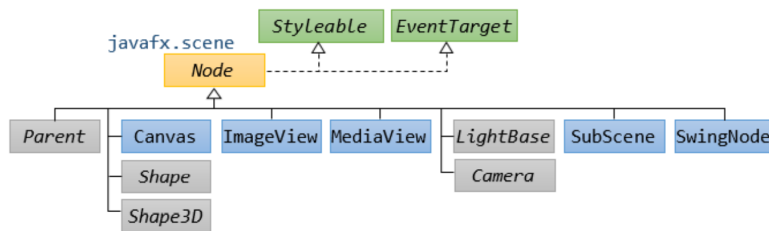


- 场景图是由节点组成的树形数据结构。
- 节点是 JavaFX 应用中的可视对象。

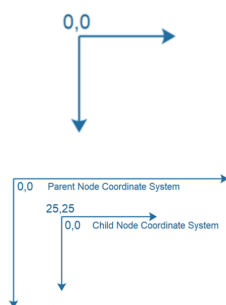
- 每个节点要么是分支节点（可包含子节点），要么是叶节点（不可包含子节点）。
- JavaFX 应用必须通过设置根属性为场景图指定根节点。
- Scene Graphs（场景图）
 - 附加到场景的所有视觉组件（控件、布局等）构成场景图。

• 图表与坐标轴

- 节点（Node）



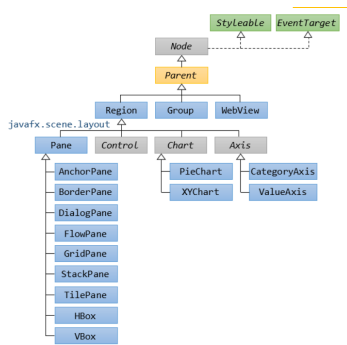
- 节点由抽象类 `javaafx.scene.Node` 定义，是所有 UI 元素的超类。
- 常见的节点子类/类型示例：
 - `Parent`（容器）
 - `Canvas`、`ImageView`、`MediaView`（绘图/媒体视图）
 - `Shape`、`Shape3D`（图形）
 - `LightBase`、`Camera`（光照/相机）
 - `SubScene`、`SwingNode` 等
- JavaFX 节点坐标系 coordinate system



- 每个 JavaFX 节点都有自己的坐标系。
- 与常见的笛卡尔坐标不同，Y 轴方向通常是向下（Y 轴方向与常规数学坐标系相反）。
- 使用坐标为子节点在父节点内定位（参见 `layoutX`, `layoutY`）。
- JavaFX 节点属性（Node Property）
 - 节点具有许多可读/可写的属性，用于控制视觉和行为。
 - 常见可写属性示例：
 - 位置： `layoutX`, `layoutY`
 - 大小： `width`, `height`（或由布局管理）

- 可见性/不透明度: visible, opacity
- 文本/内容: text (针对文本节点)
- 子节点集合: children
- 事件处理器属性: onMouseClicked 等
- 变换属性: rotate, scaleX, scaleY 等
- JavaFX 节点事件处理器属性 (EventHandler Property)
 - 节点包含多种事件处理器属性, 可以通过对应的 setter 方法设置用户自定义的事件处理器。
 - Setter 命名约定示例:
 - setOnTargetType(EventHandler<TargetEvent> v), 例如 setOnMouseClicked(...)
 - 常见事件示例: 鼠标事件 (onMouseClicked, onMouseDragged 等)、键盘事件 (onKeyPressed、onKeyReleased) 等。
 - 这些处理器通常使用函数式接口 (lambda 或方法引用) 来实现。

- 分支节点 (Branch Node)



- 分支节点 (Parent) : 包含子节点, 由抽象类 `javafx.scene.Parent` 定义, 具有若干具体子类。
- 常见子类示例:
 - **Group**: 任何对 **Group** 应用的变换 (例如旋转)、效果或状态, 会应用到该组的所有子节点上。
 - **Region**: 所有 UI 控件和布局容器的基类 (如 **Pane**、**HBox**、**VBox** 等)。
 - **WebView**: 用于显示 HTML 内容的节点。
- 叶节点 (Leaf Node) : 不能包含子节点的可视对象 (例如某些控件、图形等)。
- UI 控件 (UI Controls)
 - **Skinnable** 接口允许将控件的外观 (皮肤) 与其行为和逻辑分离。
 - 控件层次结构示例:
 - **Control** (控件) 继承自 **Region**, 是可被设置皮肤的可视组件基类。
 - 常见控件包括: **Label**、**Button**、**CheckBox**、**RadioButton**、**TextField**、**TextArea**、**ChoiceBox**、**MenuBar**、**ScrollPane**、**TabPane** 等。
 - 这种分离使得可以定制控件外观而不必改变其内部行为。
- JavaFX 图表

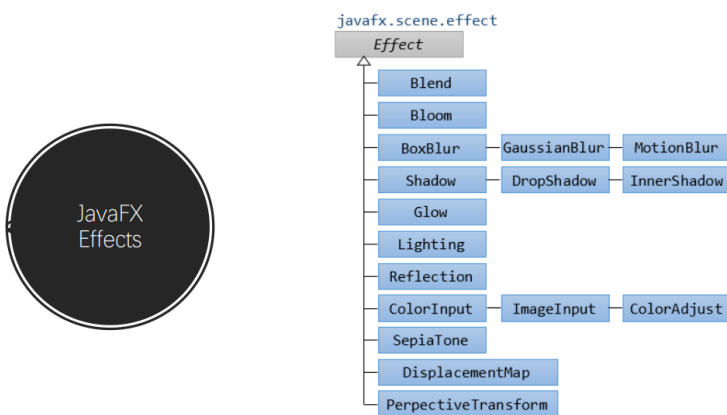
- 图表：以符号形式对数据进行图形表示的控件。
- JavaFX 图表 (javafx.scene.chart.Chart) 是所有图表的基类。
- 图表通常由三部分组成：标题、图例 (Legend) 和图表内容 (chartContent)。

• 变换、动画、效果

• JavaFX 变换 (Transformation)

- 所有变换由包 javafx.scene.transform 中的各种类表示。
- Transform 是不同变换的基类，常见变换包括：平移 (Translate)、旋转 (Rotate)、缩放 (Scale)、错切 (Shear)、仿射 (Affine) 等。
- 示例：

```
Rectangle rect = new Rectangle(50,50, Color.RED);
rect.getTransforms().add(new Rotate(45,0,0)); // 旋转 45 度
```



• JavaFX 动画

- Animation (动画) 类 (javafx.animation.Animation) :
 - 直接已知子类：Timeline、Transition
- Transition (过渡) 类 (javafx.animation.Transition) :
 - 常见具体过渡：FadeTransition (淡入/淡出)、FillTransition (填充过渡)、ParallelTransition (并行动画)、PathTransition (沿路径移动)、PauseTransition (暂停)、RotateTransition (旋转)、ScaleTransition (缩放)、SequentialTransition (顺序过渡)、StrokeTransition (描边过渡)、TranslateTransition (平移) 等

• FXML

- 设计动机：
 - 设计代码（外观）与应用代码（事件处理与逻辑）混杂在一起不利于维护
 - 如果将界面设计与应用逻辑分离，代码更易维护
 - JavaFX FXML：
 - 一种基于 XML 的语言
 - 允许将用户界面（UI）与应用逻辑分离构建
- FXML 结构
 - 一个 .fxml 文件用于设计用户界面（视图）
 - 一个控制器类用于实现应用的逻辑（事件处理、与视图交互）
 - 将视图与控制逻辑分离，便于协作和维护

Bootstrap JavaFX Application

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<?import javafx.scene.control.Button?>
<VBox alignment="CENTER" spacing="20.0" xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.example.cs209a_lectures_javaafx.HelloController">
    <padding>
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
    </padding>

    <Label fx:id="welcomeText"/>
    <Button text="Hello!" onAction="#onHelloButtonClick"/>
</VBox>
```

hello-view.fxml

```
import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class HelloController {
    @FXML
    private Label welcomeText;

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }
}
```

HelloController.java

- 在 FXML 文件（例如 hello-view.fxml）中定义布局、控件并指定 fx:controller，例如：
 - 为控件指定 fx:id（例如 Label 的 fx:id="welcomeText"）
 - 在控件上使用 onAction="#onHelloButtonClick" 关联事件处理方法
- 在控制器类（例如 HelloController.java）中：
 - 使用 @FXML 注解注入控件字段（例如 @FXML private Label welcomeText;）
 - 实现事件处理方法（例如 protected void onHelloButtonClick() { welcomeText.setText("..."); }）
- 在应用中通过 FXMLLoader 加载 .fxml 并启动界面，使视图与控制器连接运行
- JavaFX 中的多线程
 - JavaFX 线程
 - JavaFX 的 UI 只能从 JavaFX 应用线程（UI 线程）访问和修改。
 - JavaFX Scene 与 Stage 对象的创建，以及对场景图的修改，必须在 JavaFX 应用线程上完成。

```

public class Concurrency extends Application {
    @Override
    public void start(Stage primaryStage) {

        ProgressBar progressBar = new ProgressBar(0);
        VBox vBox = new VBox(progressBar);
        Scene scene = new Scene(vBox, 200, 200);
        primaryStage.setScene(scene);
        primaryStage.show();

        double progress = 0;
        for (int i = 0; i < 10; i++) {
            try {
                // mimic long-running task
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            progress += 0.1;
            double reportedProgress = progress;
            progressBar.setProgress(reportedProgress);
        }
    }
}

```

- 在 JavaFX 应用线程（例如 start() 中）执行耗时任务会导致界面无响应，直到任务完成。
- 在应用线程忙于执行该任务时，GUI 控件无法响应鼠标点击、鼠标移动或键盘输入等事件。

```

public class Concurrency extends Application {
    @Override
    public void start(Stage primaryStage) {

        ProgressBar progressBar = new ProgressBar(0);
        VBox vBox = new VBox(progressBar);
        Scene scene = new Scene(vBox, 200, 200);
        primaryStage.setScene(scene);
        primaryStage.show();

        double progress = 0;
        for (int i = 0; i < 10; i++) {
            try {
                // mimic long-running task
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            progress += 0.1;
            double reportedProgress = progress;
            progressBar.setProgress(reportedProgress);
        }
    }
}

```

- 不应在应用线程中直接运行耗时循环并在其中更新 UI（例如在循环内调用 Thread.sleep(...) 并直接调用 progressBar.setProgress(...)）。
- 这样会造成界面卡死并且导致任务实现与界面产生紧耦合。

```

@Override
public void start(Stage primaryStage) {

    ProgressBar progressBar = new ProgressBar(0);
    VBox vbox = new VBox(progressBar);
    Scene scene = new Scene(vbox, 200, 200);
    primaryStage.setScene(scene);
    primaryStage.show();

    Thread taskThread = new Thread(() -> {
        double progress = 0;
        for(int i=0; i<10; i++){
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            progress += 0.1;
            final double reportedProgress = progress;

            Platform.runLater(() -> progressBar
                .setProgress(reportedProgress));
        }
    });

    taskThread.start();
}

```

- 可以将耗时任务放到一个或多个后台线程中执行。
- 后台线程应在需要更新 UI 时，通过 Platform.runLater(...) 通知 JavaFX 应用线程执行界面更新。
- Platform.runLater 接受一个 Runnable，该 Runnable 会由 JavaFX 应用线程在合适时机执行，从而安全地修改场景图（例如更新 progressBar）。
- JavaFX Task —— 辅助方法与绑定

```

public void start(Stage primaryStage) {
    ProgressBar progressBar = new ProgressBar(0);
    Label statusLabel = new Label("Task ongoing");
    // .....

    Task<Void> task = new Task<>() {
        @Override
        protected Void call() throws Exception {
            for (int i = 0; i <= 10; i++) {
                Thread.sleep(200);
                updateProgress(i, 10);
            }
            return null;
        }
    };

    @Override
    protected void succeeded() {
        super.succeeded();
        statusLabel.setText("Task finished!");
        progressBar.setStyle("-fx-accent: green");
    }
};

progressBar.progressProperty().bind(task.progressProperty());

new Thread(task).start();
}

```

- Task 类来自 JavaFX 并发 API (javafx.concurrent)，用于运行后台操作。

- 用法：重写 call() 方法，在该方法中执行耗时的后台工作，从而不会冻结 UI。
- 示例流程：创建 Task，绑定进度属性到进度条
(progressBar.progressProperty().bind(task.progressProperty()))，在新线程中启动任务 (new Thread(task).start())。

```
public void start(Stage primaryStage) {
    ProgressBar progressBar = new ProgressBar(0);
    Label statusLabel = new Label("Task ongoing");
    // .....

    Task<Void> task = new Task<>() {
        @Override
        protected Void call() throws Exception {
            for (int i = 0; i <= 10; i++) {
                Thread.sleep(200);
                updateProgress(i, 10);
            }
            return null;
        }

        @Override
        protected void succeeded() {
            super.succeeded();
            statusLabel.setText("Task finished!");
            progressBar.setStyle("-fx-accent: green");
        }
    };

    progressBar.progressProperty().bind(task.progressProperty());

    new Thread(task).start();
}
```

70

- 辅助方法（在 call() 内使用）：
 - updateProgress(workDone, total)：更新任务的进度值（线程安全）。
 - updateMessage(msg)：更新任务的消息值（线程安全）。
 - 这些方法是线程安全的，因此无需使用 Platform.runLater。
- 绑定（Binding）：
 - UI 控件可以绑定到 Task 的属性（如 progressProperty、messageProperty）。
 - 绑定后，当 Task 的属性变化时，UI 会自动刷新显示。

```

public void start(Stage primaryStage) {
    ProgressBar progressBar = new ProgressBar(0);
    Label statusLabel = new Label("Task ongoing");
    // .....

    Task<Void> task = new Task<>() {
        @Override
        protected Void call() throws Exception {
            for (int i = 0; i <= 10; i++) {
                Thread.sleep(200);
                updateProgress(i, 10);
            }
            return null;
        }
    };

    @Override
    protected void succeeded() {
        super.succeeded();
        statusLabel.setText("Task finished!");
        progressBar.setStyle("-fx-accent: green");
    }

    progressBar.progressProperty().bind(task.progressProperty());

    new Thread(task).start();
}

```

71

- 回调方法：当后台任务状态改变时，JavaFX 框架会自动调用这些特殊方法
 - succeeded()：任务成功完成时调用
 - failed()：任务抛出异常时调用
 - cancelled()：任务被提前取消时调用
- 这些回调在 JavaFX 应用线上运行，因此无需使用 Platform.runLater() 即可直接更新 UI
- 适用于显示结果或执行清理操作
- 示例（重写 succeeded()）：
 - 在 succeeded() 中调用 super.succeeded()，更新状态标签为“任务完成！”并改变进度条样式（例如设置为绿色）